

Floating-Point Representation and Errors

Chapter 2.

Dr. Abdalnaser Rashid

Preliminary Remarks

- Computers usually do not use base-10 arithmetic for storage or computation. Numbers that have a finite expression in one number system may have an infinite expression in another system.
- This phenomenon is illustrated when the familiar decimal number $1/10$ is converted into the binary system:

$$(0.1)_{10} = (0.0\ 0011\ 0011\ 0011\ 0011\ 0011\ 0011\ 0011\ 0011\ \dots)_2$$

- In this chapter, we explain the floating-point number system and develop basic facts about round off errors.
- Another topic is: loss of significance, which occurs when nearly equal numbers are subtracted. It is studied and shown to be avoidable by various programming techniques.

Floating-Point Representation



Section 2.1

2.1 Floating-Point Representation

- The standard way to represent a nonnegative real number in decimal form is with an integer part, a fractional part, and a decimal point between them, *for example*:

37.21829, 0.002271828.

- Another standard form, often called **normalized scientific notation**, the number is represented by a fraction multiplied by 10^n . Thus, we write

79325 as 0.79325×10^5 , **not** 0.079325×10^6
or 7.9325×10^4 or some other way.

- In the context of computer science, normalized scientific notation is also called **normalized floating-point representation**.

Normalized

5

Floating-Point Representation :

- In the decimal system, any real number x (other than zero) can be represented in normalized floating-point form as:

$$x = \pm 0.d_1d_2d_3 \dots \times 10^n$$

- where $d_1 \neq 0$ and n is an integer (positive, negative, or zero). The numbers $d_1, d_2, \dots$ are the decimal digits 0, 1, 2, ..., 9.
- Stated another way, number x can be represented in normalized floating-point decimal form as :

$$x = \pm r \times 10^n \quad (1/10 \leq r < 1)$$

number r is called **the normalized mantissa** and n the **exponent**.

- The floating-point representation in the **binary system** is similar to that in the decimal system in several ways. If $x \neq 0$, it can be written as:

$$x = \pm q \times 2^m \quad (1/2 \leq q < 1)$$

Normalized Floating-Point Representation:

- The ***mantissa*** q would be expressed as a sequence of 0 or 1 in the form $q = (0.b_1b_2b_3 \dots)_2$, where $b_1 \neq 0$. Hence, $b_1=1$ and then necessarily $q \geq 1/2$.
- A floating-point number system within a computer is similar to what we have just described, with one important difference: Every computer has only a finite word length and a finite total capacity, so only numbers with a finite number of digits can be represented.
- The real numbers that are representable in a computer are called its **machine numbers**.
- The ***important point*** here is that most real numbers cannot be represented exactly in a computer.

Normalized

7

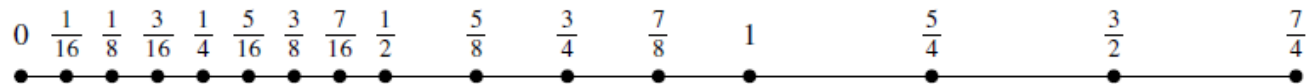
Floating-Point Representation:

- **Example 1:** List all the floating-point numbers that can be expressed in the form :

$$x = \pm (0.b_1b_2b_3)_2 \times 2^{\pm k} \quad (k, b_i \in \{0, 1\})$$

- **Solution:** The nonnegative numbers in this system are as follows:

$0.000 \times 2^0 = 0$	$0.000 \times 2^1 = 0$	$0.000 \times 2^{-1} = 0$
$0.001 \times 2^0 = \frac{1}{8}$	$0.001 \times 2^1 = \frac{1}{4}$	$0.001 \times 2^{-1} = \frac{1}{16}$
$0.010 \times 2^0 = \frac{2}{8}$	$0.010 \times 2^1 = \frac{2}{4}$	$0.010 \times 2^{-1} = \frac{2}{16}$
$0.011 \times 2^0 = \frac{3}{8}$	$0.011 \times 2^1 = \frac{3}{4}$	$0.011 \times 2^{-1} = \frac{3}{16}$
$0.100 \times 2^0 = \frac{4}{8}$	$0.100 \times 2^1 = \frac{4}{4}$	$0.100 \times 2^{-1} = \frac{4}{16}$
$0.101 \times 2^0 = \frac{5}{8}$	$0.101 \times 2^1 = \frac{5}{4}$	$0.101 \times 2^{-1} = \frac{5}{16}$
$0.110 \times 2^0 = \frac{6}{8}$	$0.110 \times 2^1 = \frac{6}{4}$	$0.110 \times 2^{-1} = \frac{6}{16}$
$0.111 \times 2^0 = \frac{7}{8}$	$0.111 \times 2^1 = \frac{7}{4}$	$0.111 \times 2^{-1} = \frac{7}{16}$



Positive machine numbers in Example 1

Normalized Floating-Point Representation:

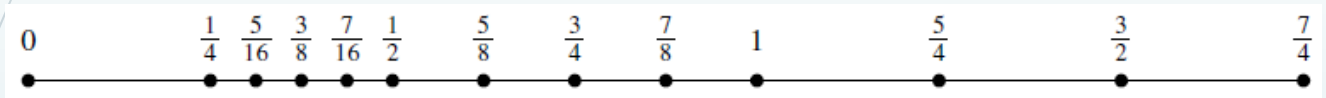
- If, in the course of a computation, a number x is produced of the form $\pm q \times 2^m$, where m is outside the computer's permissible range, then we say that an **overflow** or an **underflow** has occurred or that x is **outside the range of the computer**.
- If, in Example 1, we allow only *normalized* floating-point numbers, then all our numbers (with the exception of zero) have the form:

$$x = \pm (0.1b_2b_3)_2 \times 2^{\pm k}$$

- This creates a phenomenon known as the **hole at zero**.

Normalized Floating-Point Representation:

- Our nonnegative machine numbers are now distributed as in the figure:



- There is a relatively wide gap between zero and the smallest positive machine number, which is

$$(0.100)_2 \times 2^{-1} = 1/4 .$$

Floating-Point Representation:

- The internal representation of numbers and their storage is **standard floating-point form**, which is used in almost all computers.
- If, By **single-precision floating-point numbers**, we mean all acceptable numbers in a computer using the standard single-precision floating-point arithmetic format (*such a computer stores these numbers in 32-bit words*). This set is a finite subset of the real numbers.
- Most real numbers *cannot* be represented exactly as floating-point numbers, since they have infinite decimal or binary expansions, for example, π , e , $1/3$, 0.1 and so on.

Floating-Point Representation :

- Because of the 32-bit word-length, as much as possible of the normalized floating-point number

$$\pm q \times 2^m$$

must be contained in those 32 bits.

- One way of allocating the 32 bits is as follows:

sign of q 1 bit

integer $|m|$ 8 bits

number q 23 bits

- In such a scheme, we can represent real numbers with $|m|$ as large as $2^7 - 1 = 127$. The exponent represents numbers from -127 through 128.

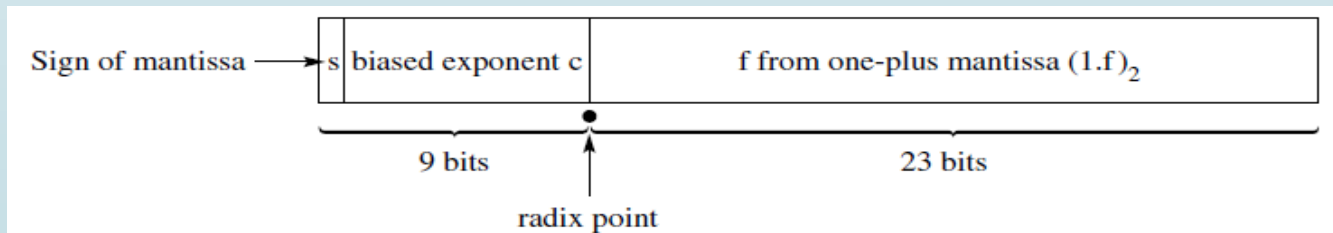
Single-Precision

Floating-Point Form :

- We now describe a machine number of the following form in **standard single-precision floating-point** representation:

$$(-1)^s \times 2^{c-127} \times (1.f)_2$$

- The leftmost bit used for the sign of the mantissa, where $s = 0$ is + and $s = 1$ is -.
- The next **eight** bits for the number c in the exponent of 2^{c-127}
- Finally, the last **23** bits represent f from the fractional part of the mantissa in the 1-plus form: $(1.f)_2$.



Single-Precision Floating-Point Form :

- The largest number representable is therefore $\approx 2^{128} \approx 3.4 \times 10^{38}$. The smallest positive number is $2^{-126} \approx 1.2 \times 10^{-38}$.
- The binary machine floating-point number $\varepsilon = 2^{-23}$ is called the **machine epsilon** when using single precision.

$$\varepsilon = 2^{-23} \approx 1.2 \times 10^{-7}$$

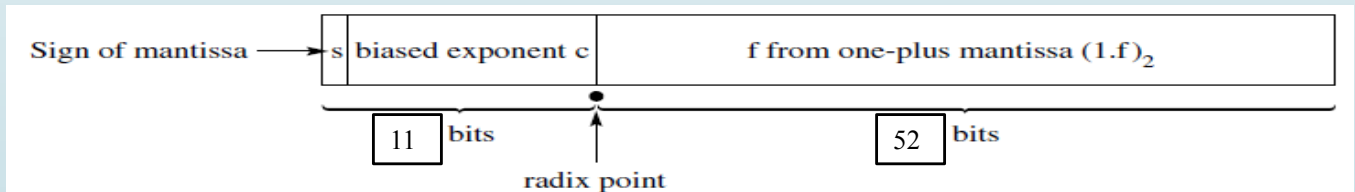
It is the smallest positive machine number ε such that $1 + \varepsilon \neq 1$.

Double-Precision Floating-Point Form :

- When more precision is needed, **double precision** can be used, in which case each double precision floating-point number is stored in two computer words in memory.
- In **standard double-precision floating-point representation**:

$$(-1)^s \times 2^{c-1023} \times (1.f)_2$$

- The leftmost bit for the sign: $s = 0$ is + and $s = 1$ is -.
- The next **eleven** bits for the number c in exponent of 2^{c-1023}



- Finally, **52** bits represent f from the fractional part of the mantissa in the one-plus form: $(1.f)_2$.

Double-Precision

15

Floating-Point Form :

- The largest double-precision machine number is: $\approx 2^{1024} \approx 1.8 \times 10^{308}$. The smallest double-precision machine number is: $2^{-1022} \approx 2.2 \times 10^{-308}$.
- The double-precision **machine epsilon** is:
$$\varepsilon = 2^{-52} \approx 2.2 \times 10^{-16}.$$

Double-Precision

16

Floating-Point Form :

- **Example :** Determine the single-precision machine representation of the decimal number -52.234375 in both single precision and double precision.
- **Solution:** Converting the integer part to binary, we have:
$$(52.)_{10} = (64.)_8 = (110\ 100.)_2.$$
- Next, converting the fractional part, we have
$$(.234375)_{10} = (.17)_8 = (.001\ 111)_2.$$
 Now :
- $(52.234375)_{10} = (110\ 100.001\ 111)_2 = (1.101\ 000\ 011\ 110)_2 \times 2^5$ is the corresponding one - plus form in base 2, and $(.101\ 000\ 011\ 110)_2$ is the stored mantissa.
- Next the exponent is $(5)_{10}$, and since $c-127 = 5$, we immediately see that $(132)_{10} = (204)_8 = (10\ 000\ 100)_2$ is the stored exponent. Thus, the single-precision machine representation of -52.234375 is

$$\begin{aligned} [1\ 10\ 000\ 100\ 101\ 000\ 011\ 110\ 000\ 000\ 000\ 00]_2 &= \\ [1100\ 0010\ 0101\ 0000\ 1111\ 0000\ 0000\ 0000]_2 &= [C250F000]_{16} \end{aligned}$$

Double-Precision

17

Floating-Point Form :

- **In double precision**, for the exponent $(5)_{10}$, we let $c-1023 = 5$, and we have $(1028)_{10} = (2004)_8 = (10\ 000\ 000\ 100)_2$, which is the stored exponent. Thus, the double-precision machine representation of $-52.23437\ 5$ is

$$\begin{aligned} [1\ 10\ 000\ 000\ 100\ 101\ 000\ 011\ 110\ 000\ \cdots 00]_2 = \\ [1100\ 0000\ 0100\ 1010\ 0001\ 1110\ 0000\ \cdots 0000]_2 = [C04A1E0000000000]_{16} \end{aligned}$$

- **Example2** : Determine the decimal numbers that correspond to these machine words:

$$[45DE4000]_{16} \quad [BA390000]_{16}$$

Refer to solution in the textbook.

Computer Errors in Representing Numbers

18

- We turn now to the errors that can occur when we attempt to represent a given real number x in the computer. We use a model computer with a 32-bit word length.
- Consider next a positive real number x in normalized floating-point form:
$$x = q \times 2^m \quad (1/2 \leq q < 1, -126 \leq m \leq 127)$$
- The process of replacing x by its nearest machine number is called **correct rounding**, and the error involved is called **roundoff error**. We want to know how large it can be.
- We suppose that q is expressed in normalized binary notation, so

$$x = (0.1b_2b_3b_4 \dots b_{24}b_{25}b_{26} \dots)_2 \times 2^m$$

Computer Errors in Representing Numbers

19

- One nearby machine number can be obtained by **rounding down** or by simply **dropping** the excess bits. This machine number is:

$$x_- = (0.1b_2b_3b_4 \dots b_{24})_2 \times 2^m$$

It lies to the left of x on the real-number axis.

- Another machine number x_+ , is just to the right of x on the real axis and is obtained by **rounding up**. It is found by adding one unit to b_{24} in the expression for x_- . Thus,

$$x_+ = [(0.1b_2b_3b_4 \dots b_{24})_2 + 2^{-24}] \times 2^m$$

- The closer of these machine numbers is the one chosen to represent x .

Computer Errors in Representing Numbers

20

- If x lies closer to x_- than to x_+ , then the relative error is bounded as follows:

$$\left| \frac{x - x_-}{x} \right| \leq \frac{2^{-25+m}}{(0.1b_2b_3b_4\dots)_2 \times 2^m} \leq \frac{2^{-25}}{\frac{1}{2}} = 2^{-24} = u$$

- Where $u = 2^{-24}$ is the **unit round off error** for a 32-bit binary computer with standard floating-point arithmetic. Moreover, $u = 2^{-k}$, ($k = 24$ in single precision and $k = 53$ in double precision).
- If x lies closer to x_+ than to x_- , then the same analysis shows that the relative error is no greater than $2^{-24} = u$.



A possible relationship between x_- , x_+ , and x .

Notation $\text{fl}(x)$ and

21

Backward Error Analysis

- **Next** let us turn to the errors that are produced in the course of elementary arithmetic operations.
- suppose that we are working with a five-place decimal machine and wish to add numbers. Two typical machine numbers in normalized floating point form are:

$$x = 0.37218 \times 10^4 \quad y = 0.71422 \times 10^{-1}$$

- Let Us assume that our computer will have a ten-place accumulator. First, the exponent of the smaller number is adjusted so that both exponents are the same. Then the numbers are added in the accumulator, and the rounded result is placed in a computer word:

$$\begin{array}{r} x = 0.37218\,00000 \times 10^4 \\ y = 0.00000\,71422 \times 10^4 \\ \hline x + y = 0.37218\,71422 \times 10^4 \end{array}$$

Notation $\text{fl}(x)$ and

22

Backward Error Analysis

- The nearest machine number is $z = 0.37219 \times 10^4$, and the relative error involved in this machine addition is:

$$\frac{|x + y - z|}{|x + y|} = \frac{0.0000028578 \times 10^4}{0.3721871422 \times 10^4} \approx 0.77 \times 10^{-5}$$

- To facilitate the analysis of such errors, it is convenient to introduce the notation **$\text{fl}(x)$** to denote the **floating-point machine number** that corresponds to the real number x .
- The hypothetical five-decimal-digit machine used above would give:

$$\text{fl}(0.3721871422 \times 10^4) = 0.37219 \times 10^4$$

Notation $\text{fl}(x)$ and

23

Backward Error Analysis

- For a 32-bit word-length computer, we established previously that if x is any real number within the range of the computer, then

$$\frac{|x - \text{fl}(x)|}{|x|} \leq u \quad (u = 2^{-24})$$

- This inequality can also be expressed in the more useful form :

$$\text{fl}(x) = x(1 + \delta) \quad (|\delta| \leq 2^{-24})$$

- This interpretation is an example of **backward error analysis**.

Summary

24

(1) A **single-precision floating-point number** in a 32-bit word-length computer with standard floating-point representation is stored in a single word with the bit pattern

$$b_1 b_2 b_3 \cdots b_9 b_{10} b_{11} \cdots b_{32}$$

which is interpreted as the real number

$$(-1)^{b_1} \times 2^{(b_2 b_3 \cdots b_9)_2} \times 2^{-127} \times (1.b_{10} b_{11} \cdots b_{32})_2$$

➤ (2) A **double-precision floating-point number** in a 64-bit word-length computer with standard floating-point representation is stored in two words with the bit pattern

$$b_1 b_2 b_3 \cdots b_9 b_{10} b_{11} b_{12} b_{13} \cdots b_{32} b_{33} b_{34} b_{35} \cdots b_{64}$$

➤ which is interpreted as the real number

$$(-1)^{b_1} \times 2^{(b_2 b_3 \cdots b_{12})_2} \times 2^{-1023} \times (1.b_{13} b_{14} \cdots b_{64})_2$$

➤ (3) The relationship between a real number x and the **floating-point machine number** $\text{fl}(x)$ can be written as

$$\text{fl}(x) = x(1 + \delta) \quad (|\delta| \leq 2^{-24})$$

➤ If $\odot$ denotes any one of the arithmetic operations, then

$$\text{fl}(x \odot y) = (x \odot y)(1 + \delta)$$

In these equations, δ depends on x and y .



END